MERGE SORT VS HEAP SORT

By: Gizelle De La Torre, Lifu Erhart, Mohamed Eljack, Henry Owen

```python
def heapify(arr, n, i):
    # Assume root is largest
    largest = i
    l = 2 * i + 1 # left child index
    r = 2 * i + 2 # right child index

    # check if left child exists and is > root
    if l < n and arr[l] > arr[largest]:
        largest = l
    # check if right child exists and is > largest
    if r < n and arr[r] > arr[largest]:
        largest = r
    # if largest is not the root, swap and heapify
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i
        heapify(arr, n, largest)
```

```python
def heapSort(arr):
    n = len(arr)

    if n <= 1:
        return

    # build max heap from array
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # move max to end, re-heapify
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
```

Project Overview

Algorithm Background

Evaluation Methods

Input Data

Visual Graphs

Conclusion

agenda

PROJECT OVERVIEW

Our project continues to take the empirical route with comparing three algorithms and their performances.

The algorithms tested are still merge sort, heap sort, and double-pop heap sort. They will continue to be tested using the same problems.

Focusing on how much memory is being used by each algorithm.

```
mid = (left + right) // 2 # find midpoint
mergeSort(arr, left, mid) # recurse on left half
mergeSort(arr, mid + 1, right) # recurse of right half
```

ALGORITHM BACKGROUND

Heap Sort

- Heap sort is based on the algorithm Binary Heap.
- Builds max or min heap by using heapify.
- The algorithm works by going through the array and looking for the maximum, or sometimes minimum, element and committing a swap with either the first or last element.
- Heap sort has a time complexity of O(n log n).

Merge Sort

- A divide-and-conquer algorithm.
- The array is recursively split into halves and to become subproblems.
- Subproblem are solved and then merged back together to get a sorted array.
- Merge sort has a time complexity of O(n log n).

Double-pop Heapsort

- A variation from the Heapsort algorithm.
- Extracting two largest elements each iteration (root node & one of the children).
- Swaps root with last element and then performs a re-heapify.
- Repeats process until the array is sorted.
- Double-pop Heapsort still maintains a time complexity O(n log n).

EVALUATION METHODS

- Ran Merge Sort, Heap Sort, and Double Pop Heap Sort on identical data
- Generated random arrays of different input sizes and distributions
- Each test was averaged over 10 trials
- Recorded time with perf_counter()
- Recorded memory allocation with tracemalloc()

INPUT DATA

Random – Average case behavior with no structural bias

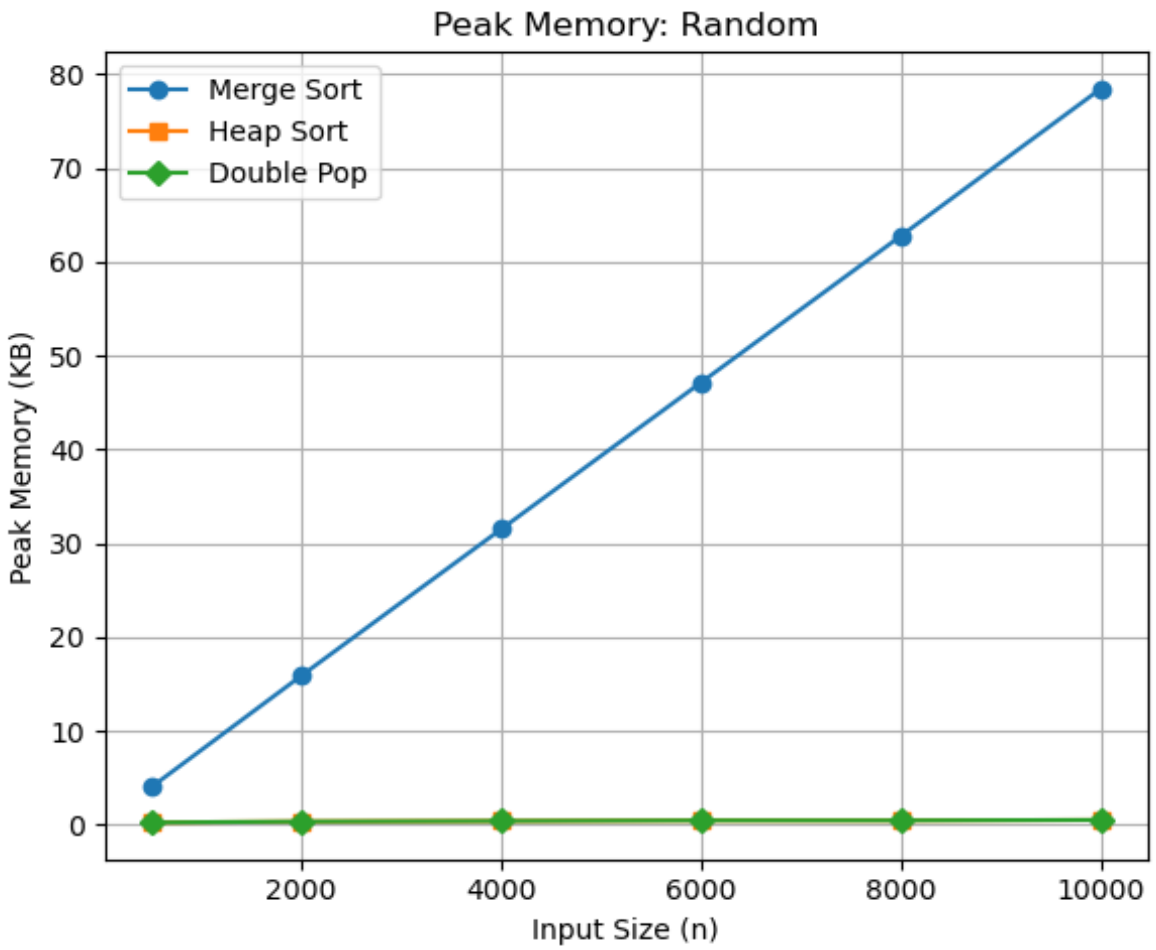
Sorted – Best case, test's Merge Sort's early exit

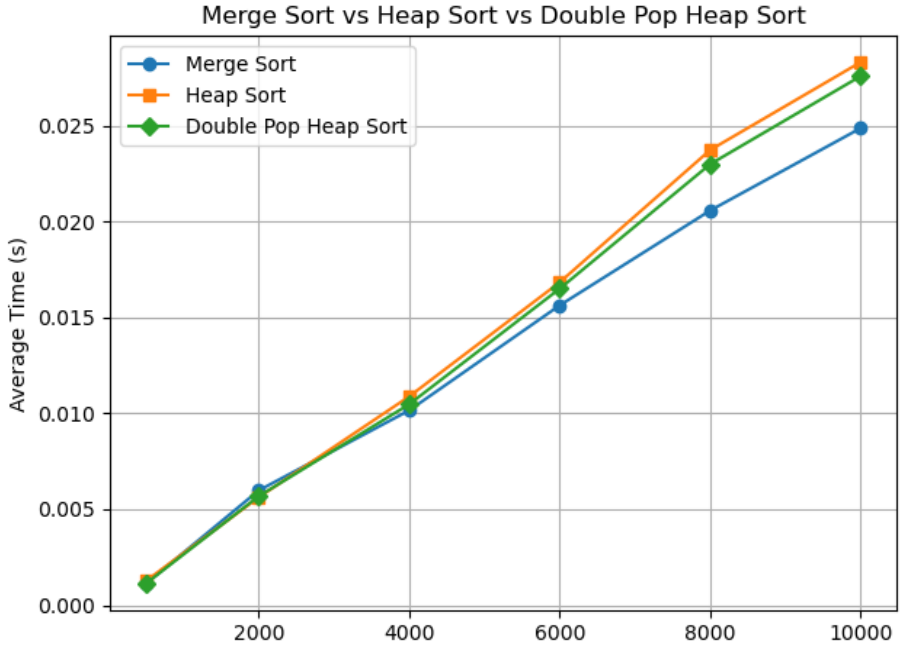
Reverse – Worst case for comparison based

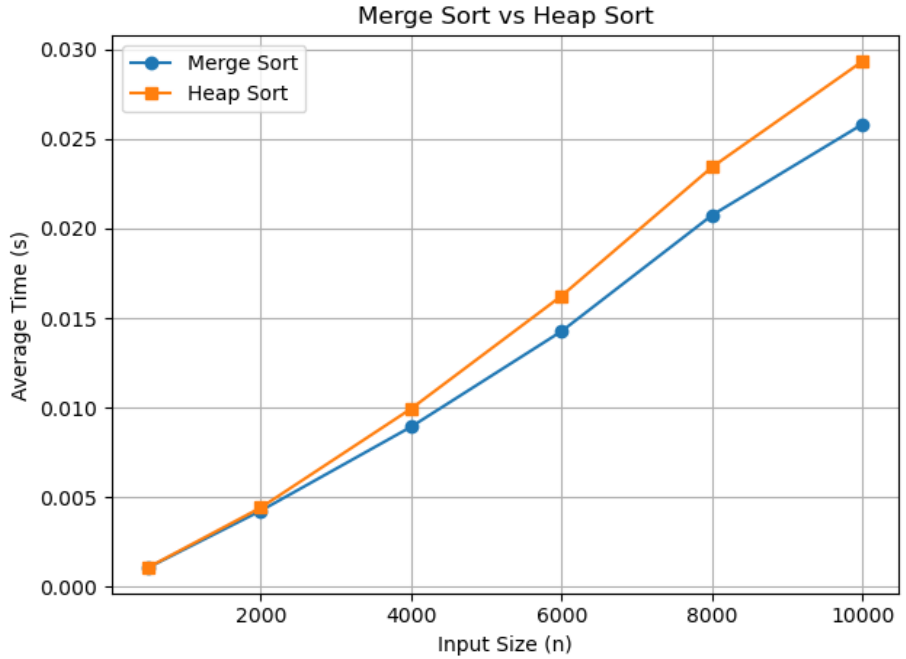
Nearly Sorted – Approx. 5% of elements are swapped, further tests early exit of merge sort

Duplicates – Validates partition balance for merge sort and heap maintenance for heap sort

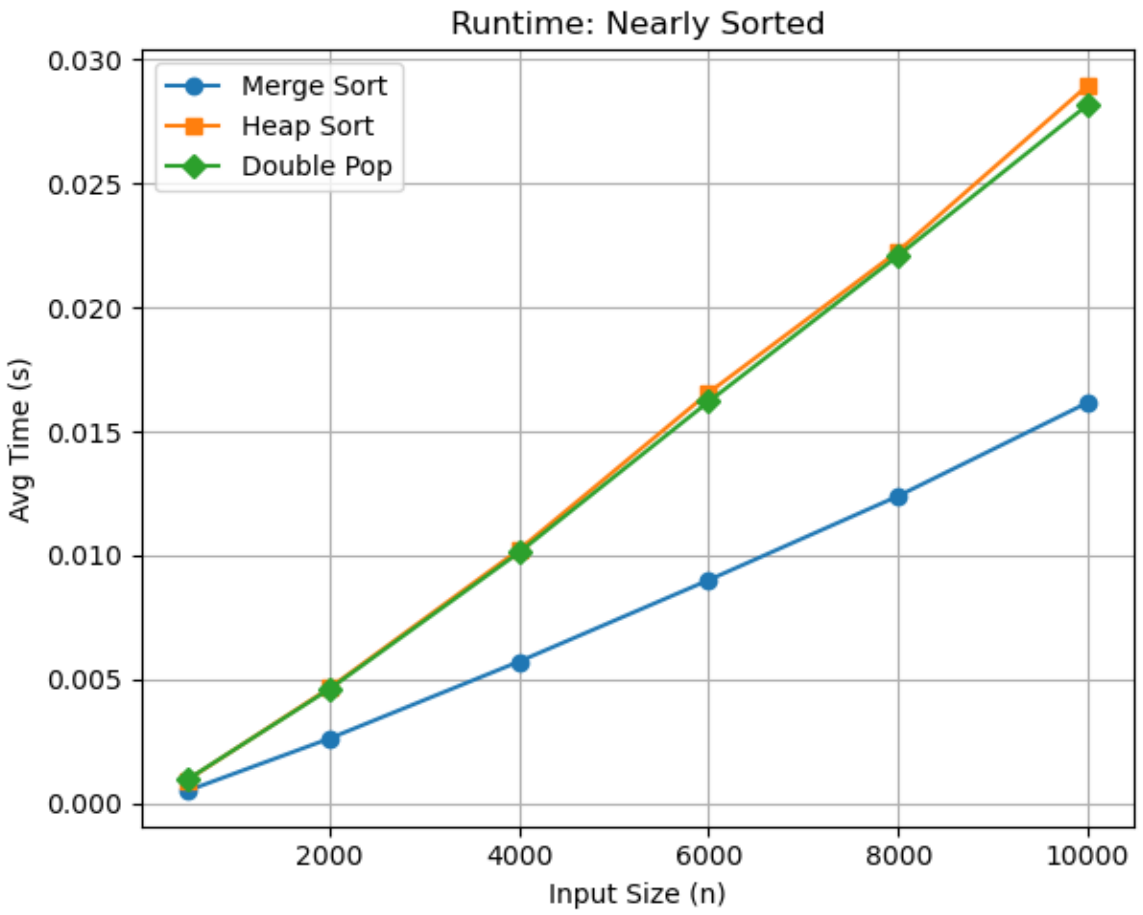
MERGE SORT VS HEAP SORT

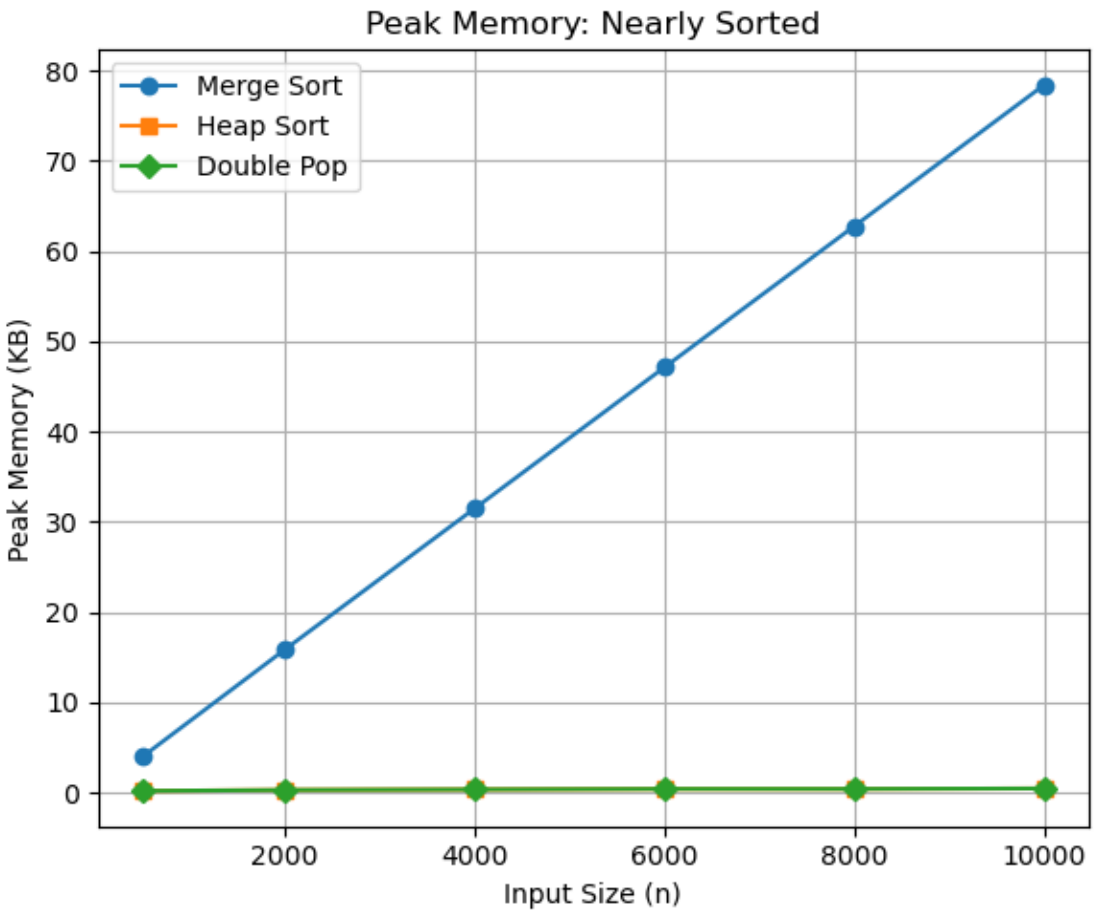






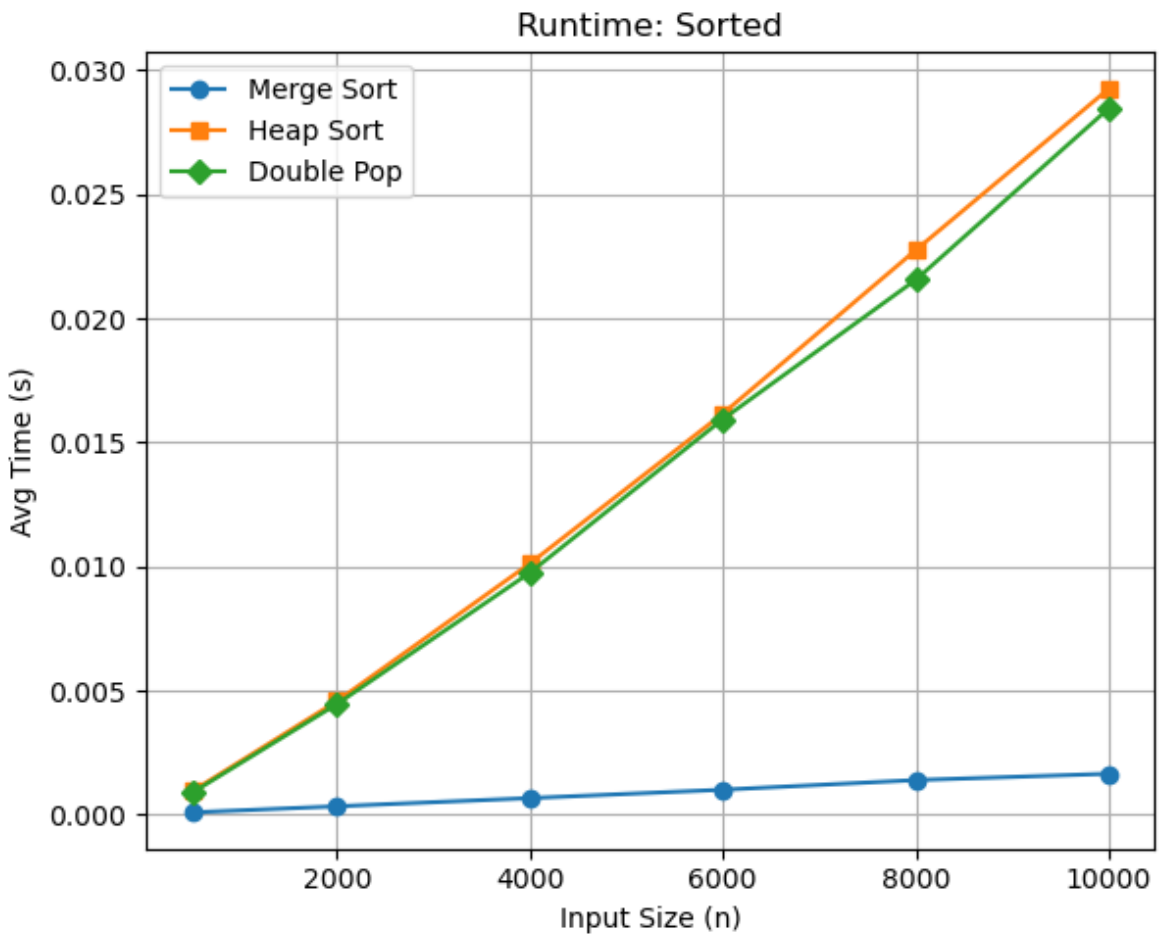
RESULTS TO NEARLY SORTED

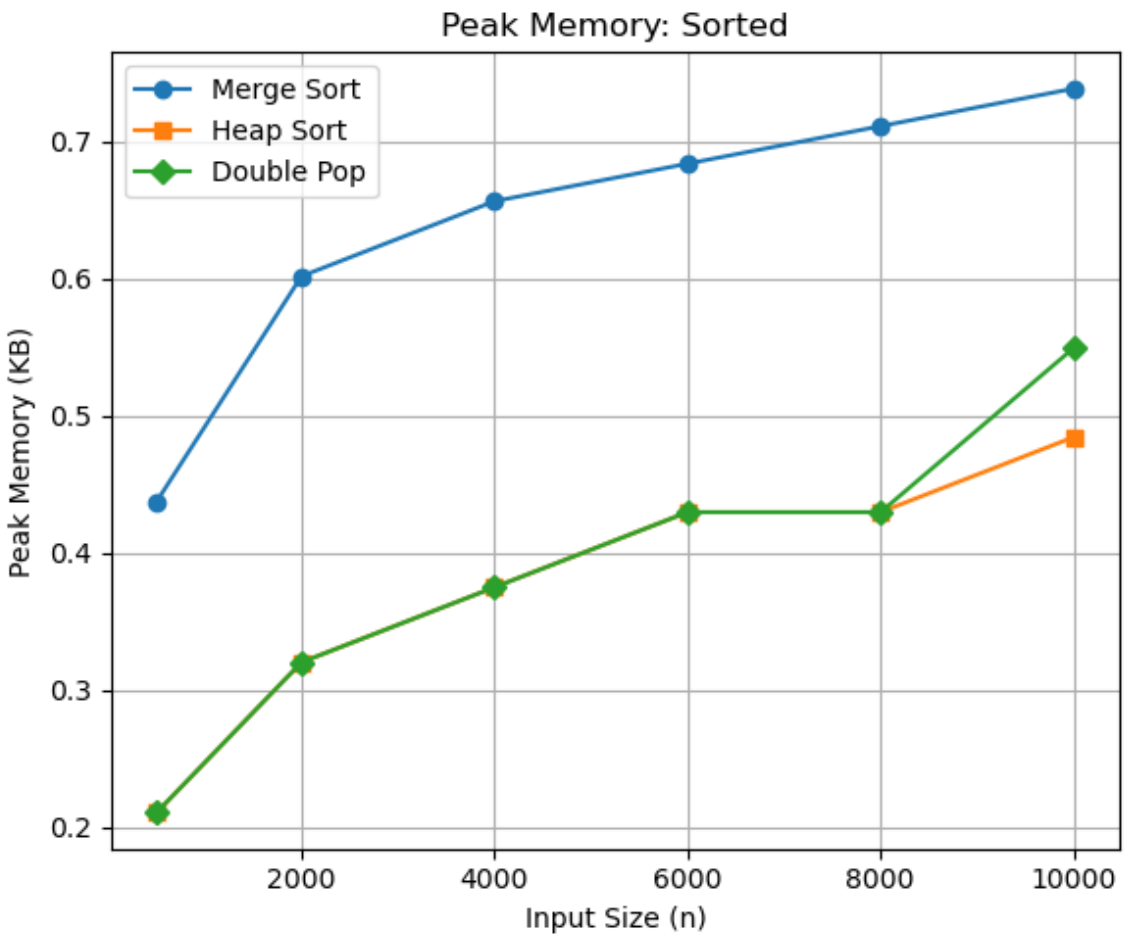




Nearly sorted data benefits Merge Sort the most in runtime.. heap sorts lag behind here.

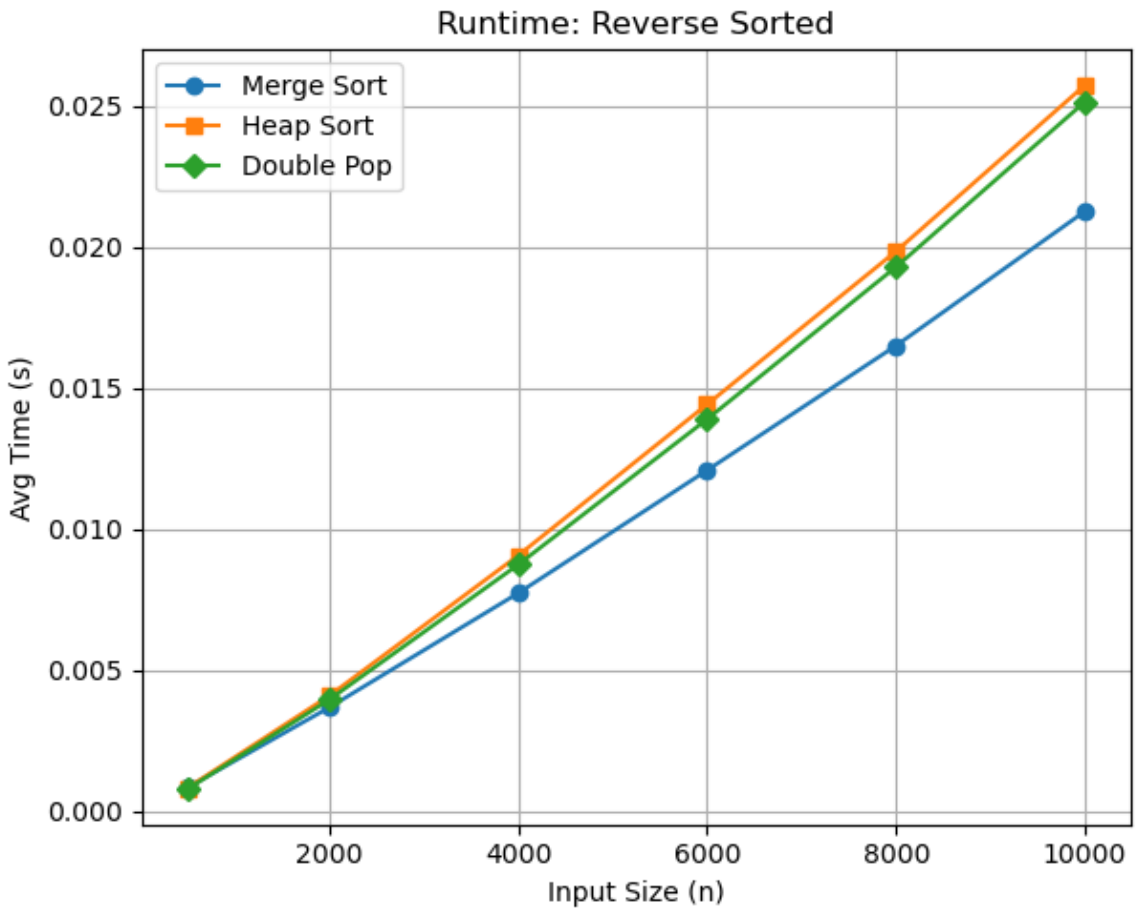
RESULTS TO ALREADY SORTED

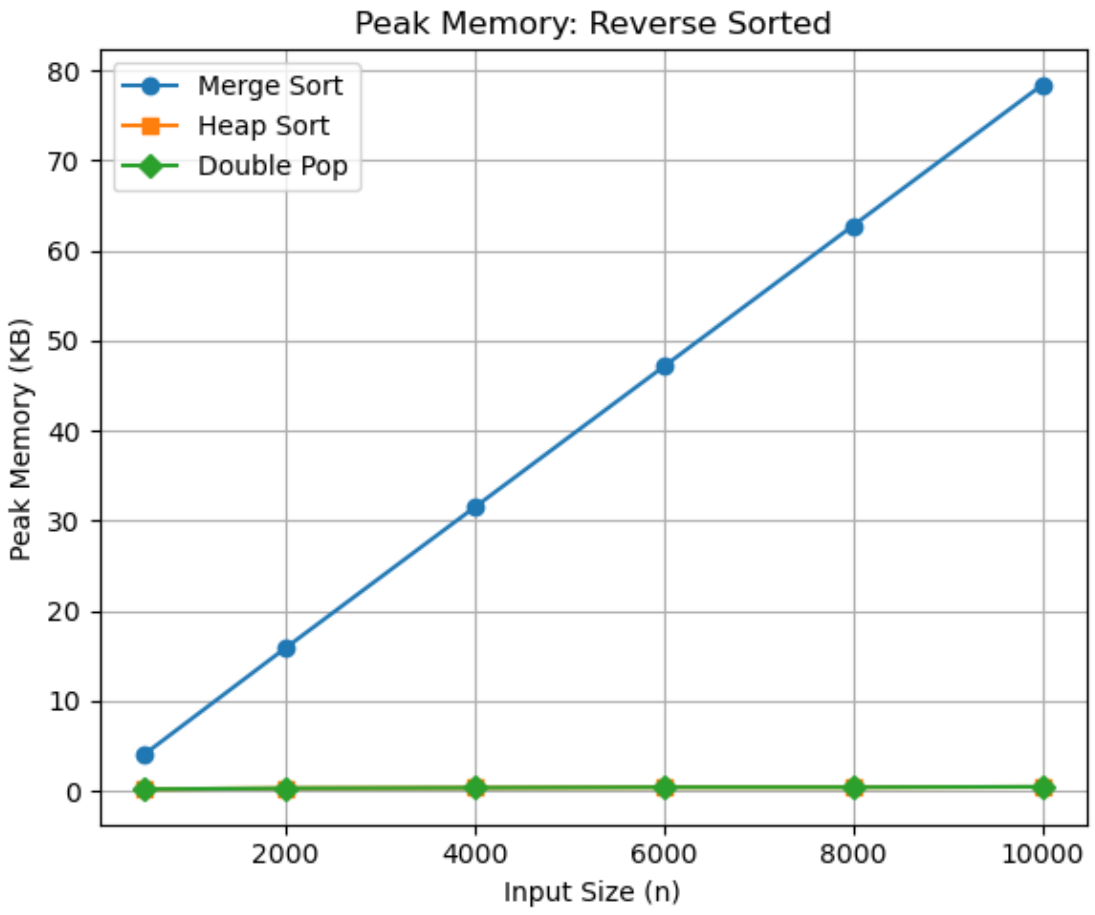




Merge Sort's early exit gives it a massive runtime advantage on sorted data, but memory cost stays high. Early exit: `if A[mid] > A[mid + 1]`

RESULTS TO REVERSE SORT





Even in worst case input, Merge Sort holds a runtime edge, while heap sorts stay memory efficient.

KEY OBSERVATIONS

- Merge Sort consistently had the lowest runtime across most input sizes, as the input size increased.

- The performance gap between Merge Sort and Heap Sort become more noticeable at larger input sizes

- Nearly sorted and sorted inputs gave Merge Sort a clear advantage, shown by a steeper drop in runtime

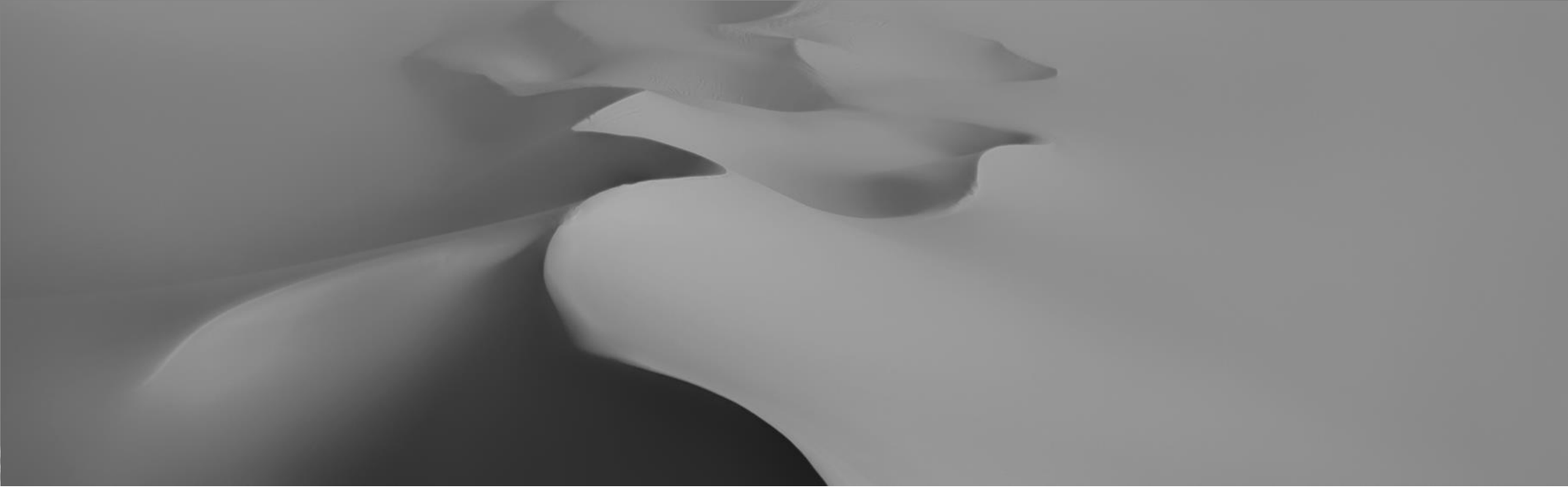
- Heap Sort and Double-Pop Heap Sort were more consistent across input types but never the fastest

- Merge Sort used significantly more memory, Showing a trade-off between speed and space

CONCLUSION

When comparing the graphs, Merge Sort consistently had lower runtime curves than Heap Sort as input size increased. This gap became more noticeable on larger datasets, showing that Merge Sort scales better in practice. However, the memory graphs showed a clear trade-off, where Merge Sort used significantly more memory compared to heap-based algorithms.

This confirms that while all algorithms have O(n log n), their real-world performance differs depending on memory vs speed.



THANK YOU